

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

Model Configuration

MobileNetV2 pretrained on ImageNet was used as the feature extractor. The MobileNetV2 layers were kept frozen, and a Global Average Pooling layer followed by a Dense classification layer with 37 output classes was used.

The model contains a total of 2,305,381 parameters, of which 47,397 are trainable and 2,257,984 are non-trainable.

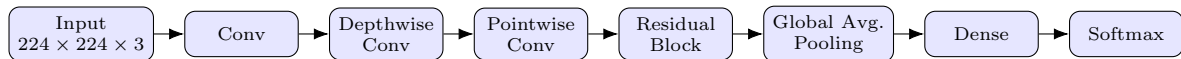
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Initialization	Final Training Loss	Best Validation Accuracy (%)	Best Epoch	Val. Acc. Improvement (%)
Zero	0.056769	91.576087	3	0.271738
Random	0.064154	90.489131	5	2.989131
Xavier/Glorot	0.060967	90.760869	4	0.951082
He	0.063061	91.983694	5	2.309781

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Training Loss
- Plot one curve for each initialization method.

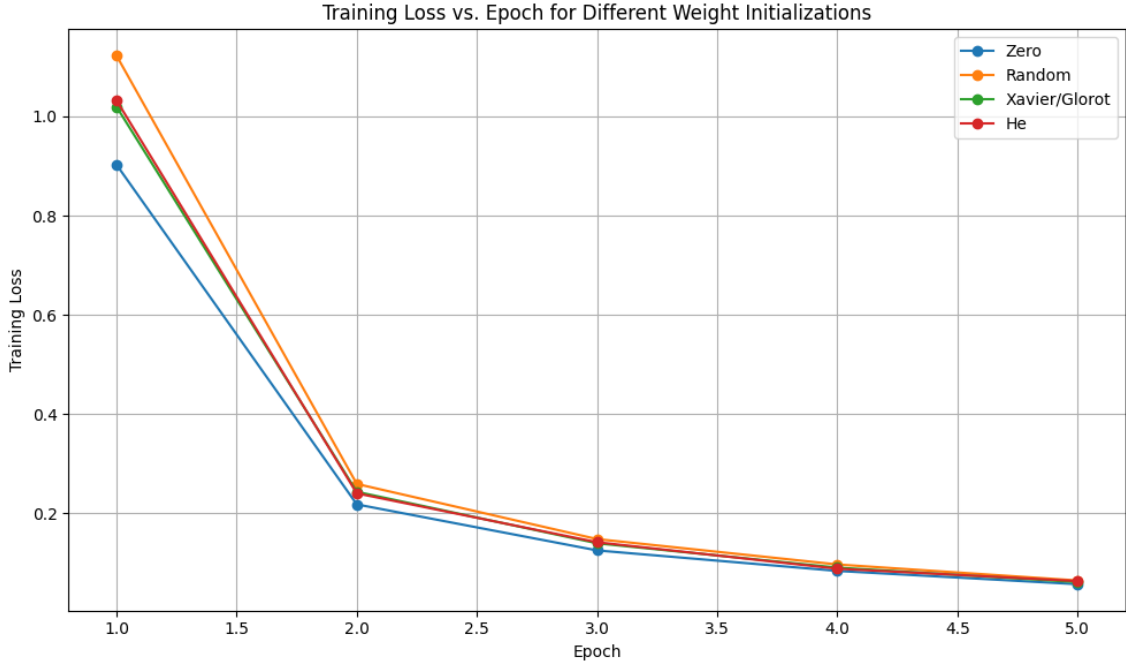


Figure 1: Training Loss vs. Epoch for Different Weight Initialization Methods

Inference: The plot shows the variation of training loss across epochs for Zero, Random, Xavier/Glorot, and He initialization methods. The training loss generally decreases as the number of epochs increases, with He initialization showing effective and stable convergence. This trend occurs because a suitable weight initialization provides appropriate starting weights, allowing the optimization process to update the model parameters more effectively.

Plot 2: Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Plot one curve for each initialization method.

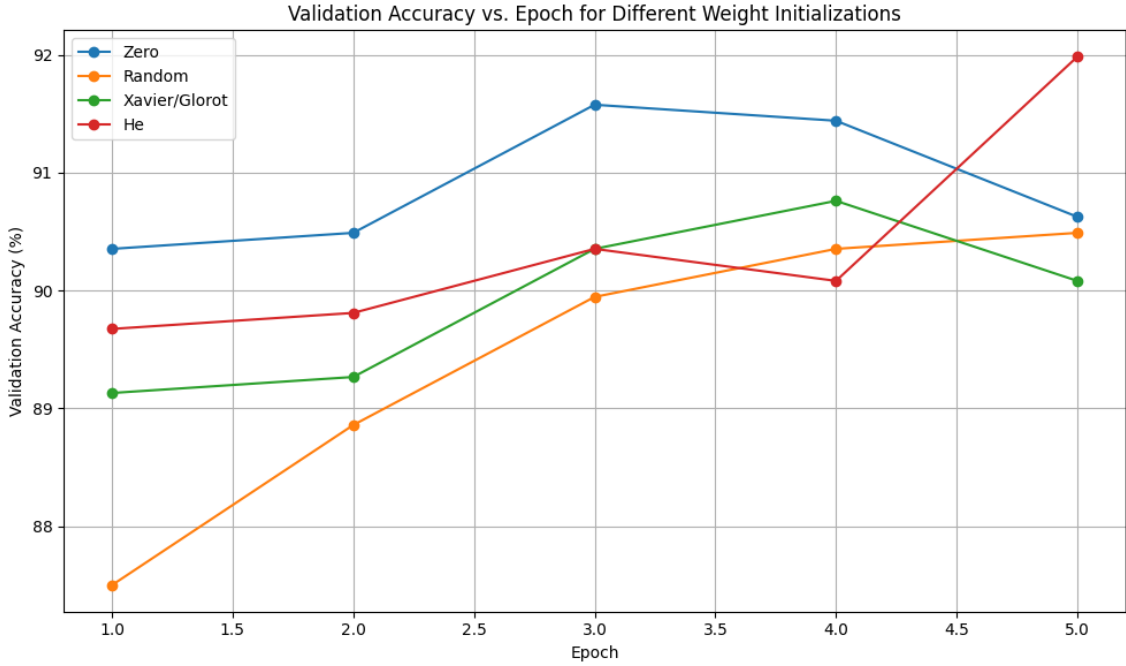


Figure 2: Validation Accuracy vs. Epoch for Different Weight Initialization Methods

Inference: The plot shows the validation accuracy obtained using Zero, Random, Xavier/Glorot, and He initialization across the training epochs. He initialization achieves the highest validation accuracy of 91.98% and shows stable convergence compared with the other methods. This may occur because He initialization provides suitable initial weight scaling for nonlinear activations, allowing the model to learn effective features more efficiently.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Method	Final Train Acc. (%)	Final Val. Acc. (%)	Gap (%)	Best Val. Acc. (%)	Best Epoch	Final Train Loss	Final Val. Loss
No Regularization	99.558425	91.168481	8.389944	91.168481	5	0.060255	0.297296
L2 Regularization	99.388587	90.217394	9.171194	90.489131	4	0.188222	0.399089
Dropout	95.006794	90.489131	4.517663	91.168481	4	0.173856	0.299376
Batch Normalization	98.845106	91.168481	7.676625	91.168481	5	0.050831	0.324522

Plot 3: Training and Validation Accuracy vs. Epoch

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Plot separate curves for training and validation accuracy.

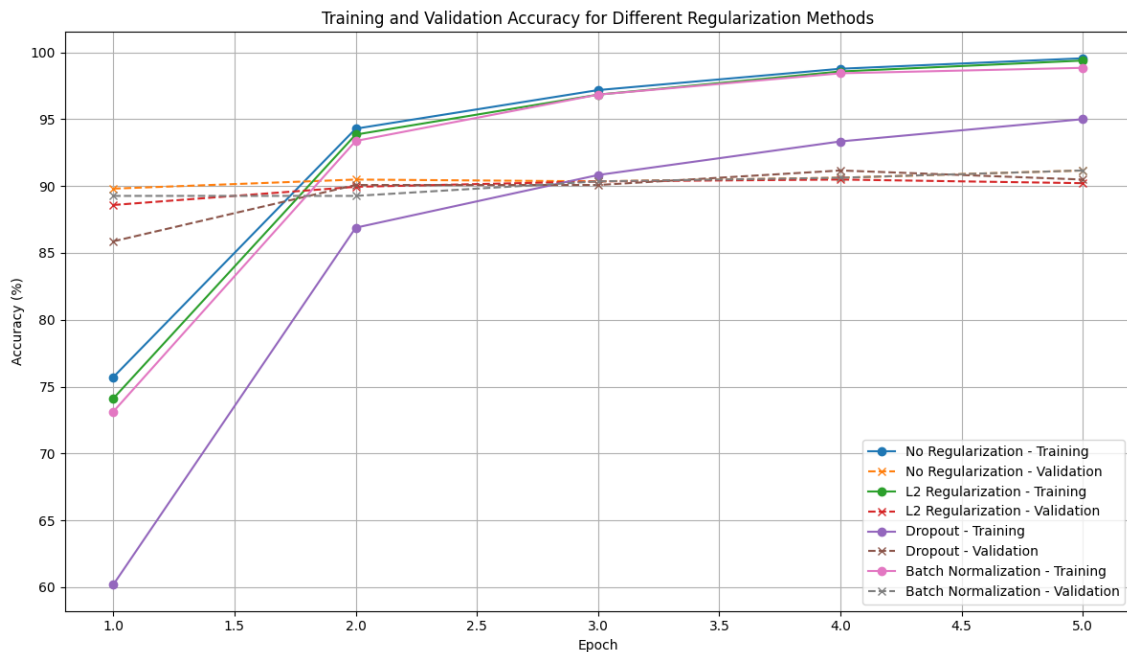


Figure 3: Training and Validation Accuracy vs. Epoch

Inference: The plot shows the training and validation accuracy across epochs and helps observe the generalization gap. Training accuracy increases as the model learns, while validation accuracy improves at a slower rate, creating a small gap between the two curves. This gap may occur because the model learns the training samples more effectively than unseen validation samples.

Plot 4: Training and Validation Loss vs. Epoch

- X-axis: Epoch
- Y-axis: Loss
- Plot training and validation loss.

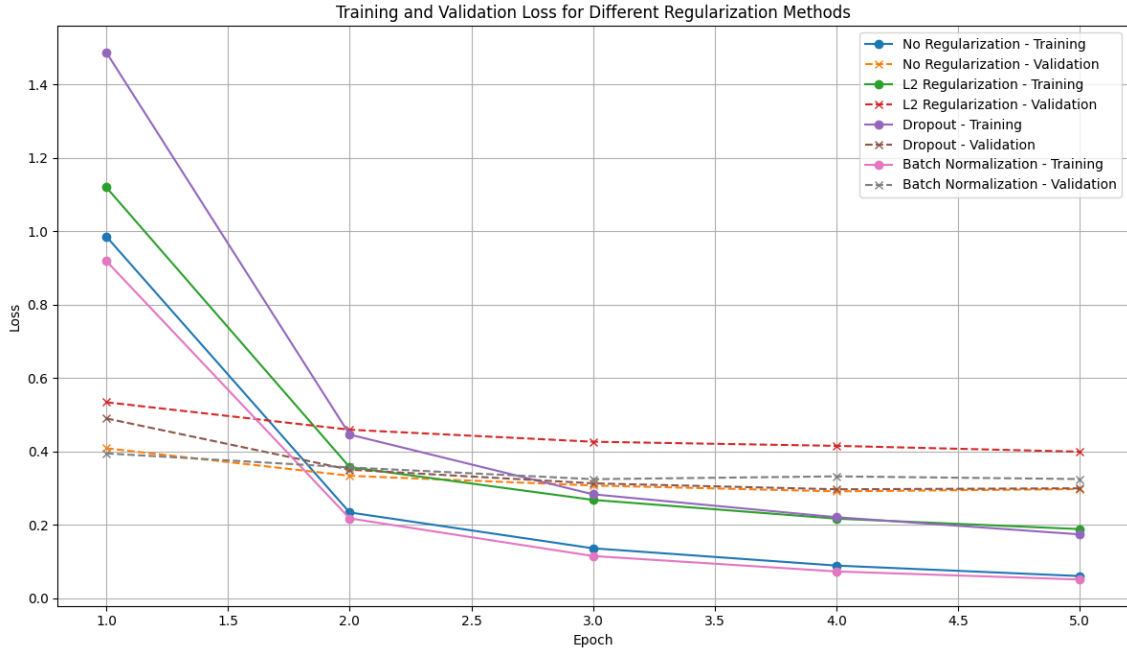


Figure 4: Training and Validation Loss vs. Epoch

Inference: The plot shows how training and validation loss change across epochs. Both losses generally decrease as training progresses, without a strong increase in validation loss while training loss decreases. This indicates that severe overfitting is not observed within the training period, as the validation performance remains relatively stable.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2-5)^2 + (4-5)^2 + (6-5)^2 + (8-5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2-5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4-5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6-5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8-5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Method	Best Validation Accuracy (%)	Best Epoch
Without BN	90.90	5
With BN	90.35	3

Plot 5: With vs. Without Batch Normalization

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Compare two curves: With BN and Without BN.

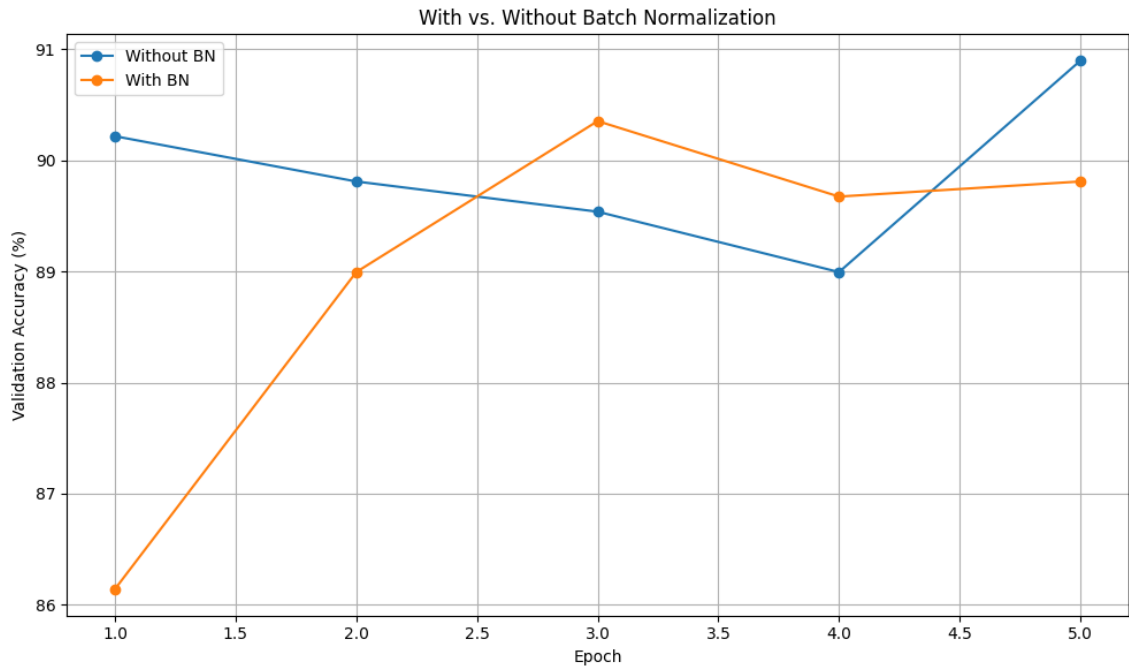


Figure 5: Validation Accuracy With and Without Batch Normalization

Inference: The plot compares the validation accuracy of the model with and without Batch Normalization across epochs. The model without Batch Normalization achieves slightly higher validation accuracy than the model with Batch Normalization in this experiment. This may indicate that Batch Normalization does not provide an additional benefit for the particular frozen MobileNetV2 feature representation used here.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	1.883670	70.244563%	5	91.255665 s
Momentum	0.318017	89.673913%	5	85.150830 s
RMSProp	0.067855	90.353262%	4	92.284208 s
Adam	0.062630	91.711956%	5	90.548798 s

Plot 6: Training Loss vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Training Loss
- One curve for each optimizer.

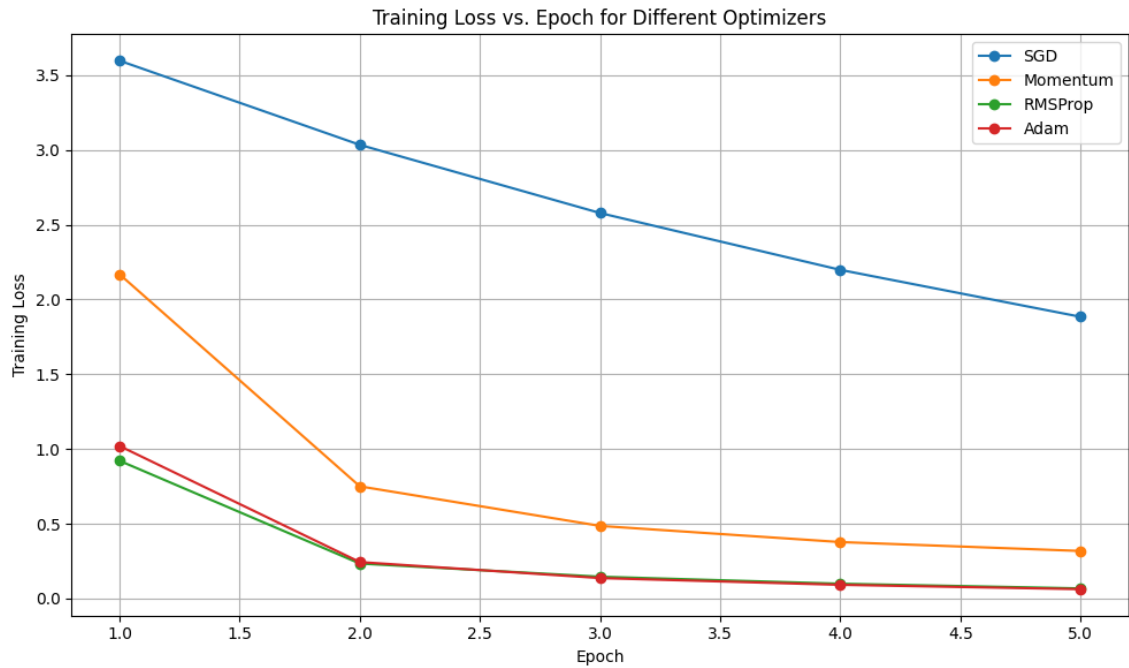


Figure 6: Training Loss vs. Epoch for Different Optimizers

Inference: The plot shows the training loss across epochs for SGD, Momentum, RMSProp, and Adam. The loss decreases for all optimizers, but Adam shows faster and more effective reduction in training loss compared with the other methods. This may occur because Adam uses adaptive learning rates for individual parameters, allowing efficient parameter updates during optimization.

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- One curve for each optimizer.

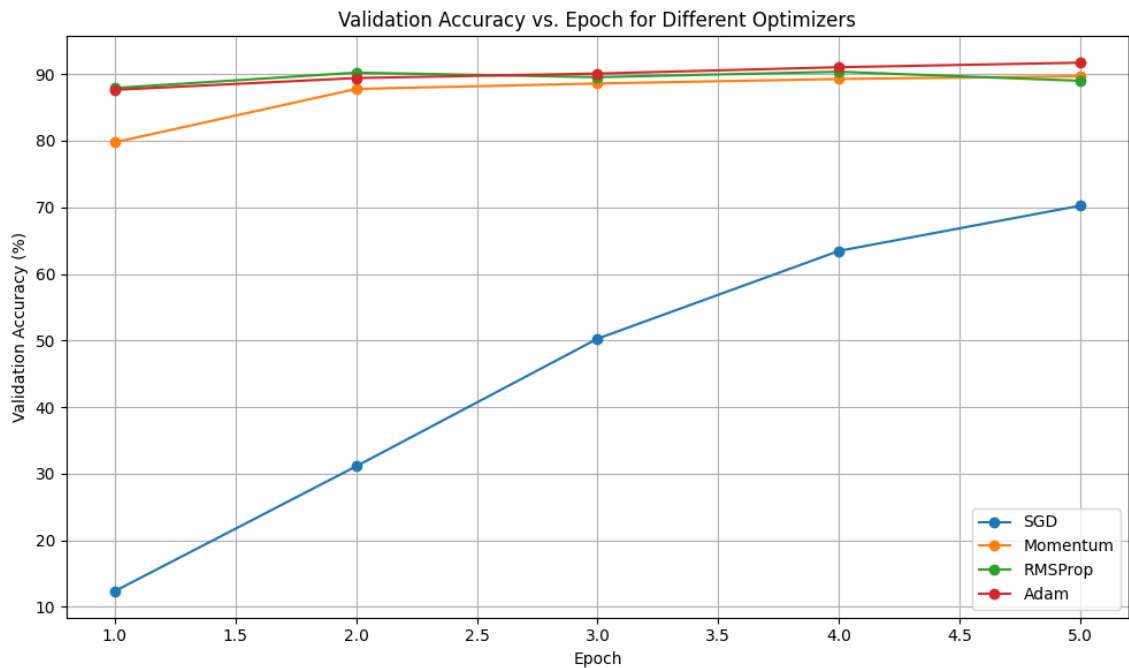


Figure 7: Validation Accuracy vs. Epoch for Different Optimizers

Inference: The plot compares the validation accuracy obtained using SGD, Momentum, RMSProp, and Adam over the training epochs. Adam achieves the highest validation accuracy of approximately 91.71% and shows effective convergence compared with the other optimizers. Its adaptive parameter updates can help the model reach a good solution more quickly and stably.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy

- X-axis: Learning Rate
- Y-axis: Validation Accuracy (%)

Learning Rate	Best Validation Accuracy (%)	Best Epoch	Time (seconds)
0.0010	90.353262	3	57.784095
0.0001	80.570650	3	56.810362

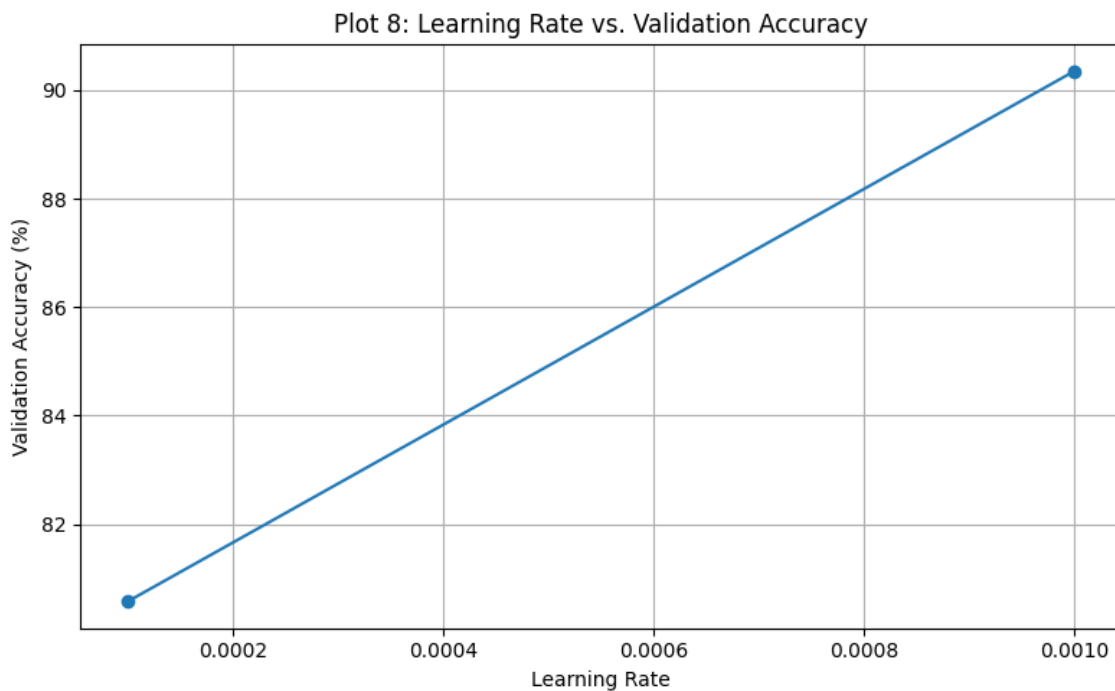


Figure 8: Learning Rate vs. Validation Accuracy

Inference: The plot shows the effect of learning rate on validation accuracy. A learning rate of 0.001 achieved 90.353262% validation accuracy, while 0.0001 achieved 80.570650%. The higher learning rate therefore performed better in this experiment, indicating that it allowed more effective parameter updates within the given training epochs.

Plot 9: Batch Size vs. Validation Accuracy

- X-axis: Batch Size
- Y-axis: Validation Accuracy (%)

Batch Size	Best Validation Accuracy (%)	Best Epoch	Time (seconds)
16	90.489131	3	58.811869
32	90.625000	3	68.616273
64	90.081519	3	70.545443

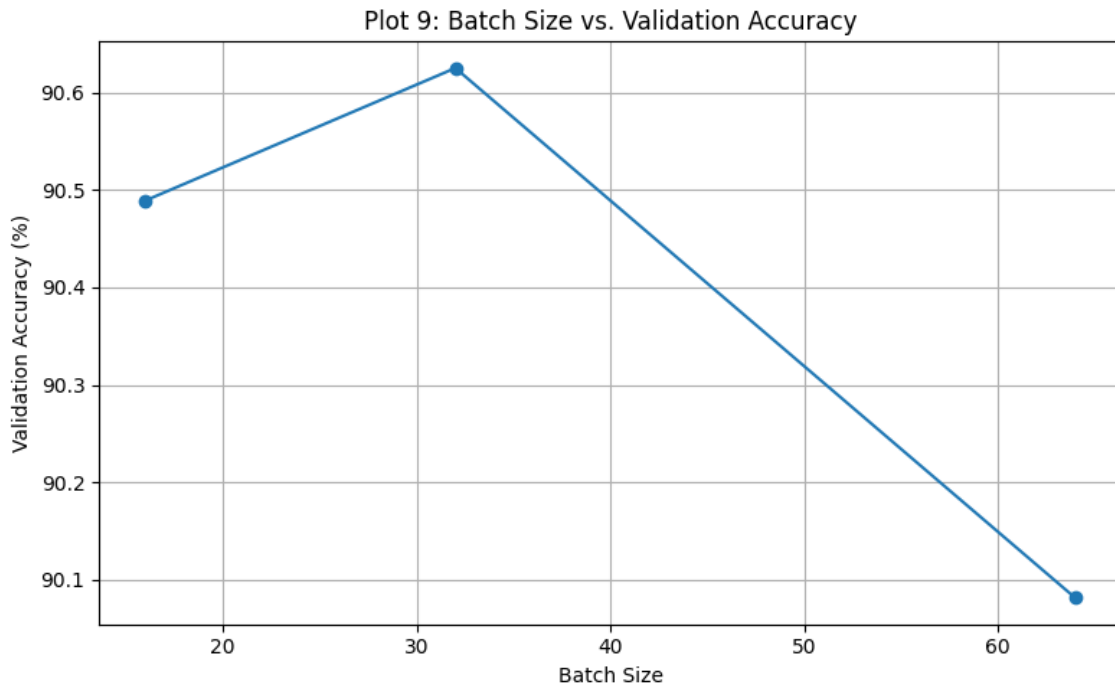


Figure 9: Batch Size vs. Validation Accuracy

Inference: The plot shows the effect of batch size on validation accuracy for batch sizes 16, 32, and 64. Batch size 32 achieved the highest validation accuracy of 90.625000%, followed by 16 with 90.489131% and 64 with 90.081519%. This suggests that the intermediate batch size provided the most effective balance between stable and frequent parameter updates in this experiment.

Plot 10: Dropout Rate vs. Validation Accuracy

- X-axis: Dropout Rate
- Y-axis: Validation Accuracy (%)

Dropout Rate	Best Validation Accuracy (%)	Best Epoch	Time (seconds)
0.00	89.809781	2	60.878376
0.25	90.625000	2	59.674658
0.50	89.945650	3	64.191078

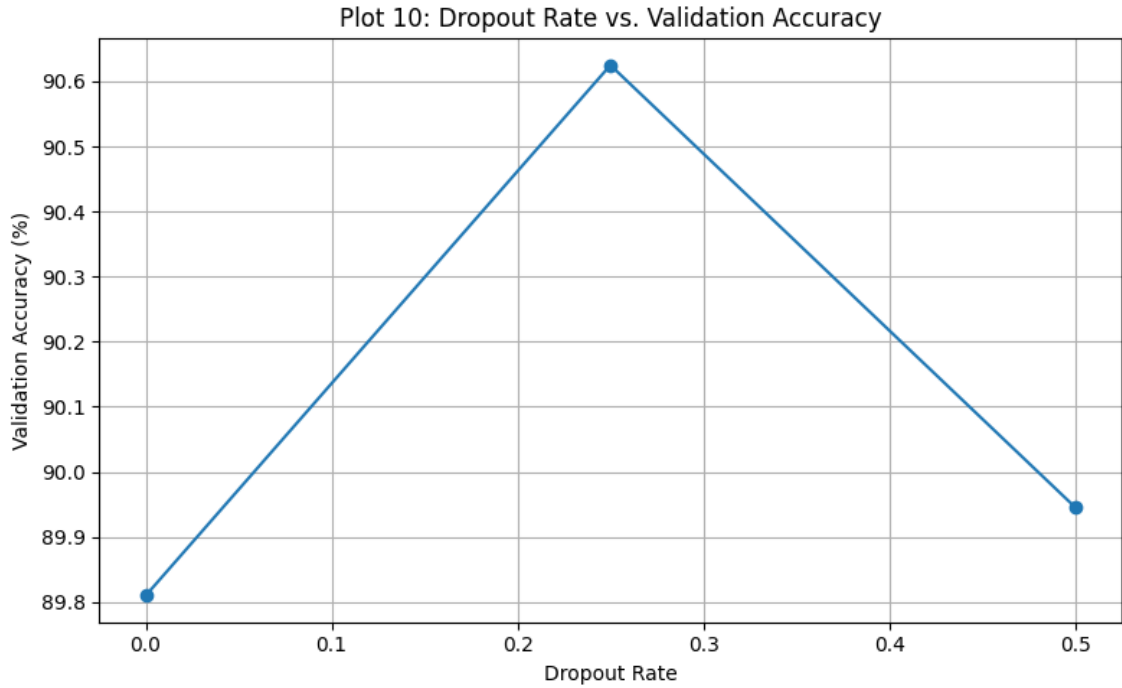


Figure 10: Dropout Rate vs. Validation Accuracy

Inference: The plot shows the effect of dropout rate on validation accuracy. A dropout rate of 0.25 achieved the highest validation accuracy of 90.625000%, compared with 89.809781% for 0.00 and 89.945650% for 0.50. The moderate dropout rate provided some regularization while retaining sufficient information for effective learning.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Method	Best Validation Accuracy (%)	Final Training Loss
Feature Extraction	90.896738	0.140670
Fine-Tuning	76.766306	1.568698

Plot 11: Feature Extraction vs. Fine-Tuning

- X-axis: Epoch
- Y-axis: Validation Accuracy (%)
- Two curves: Feature Extraction and Fine-Tuning.

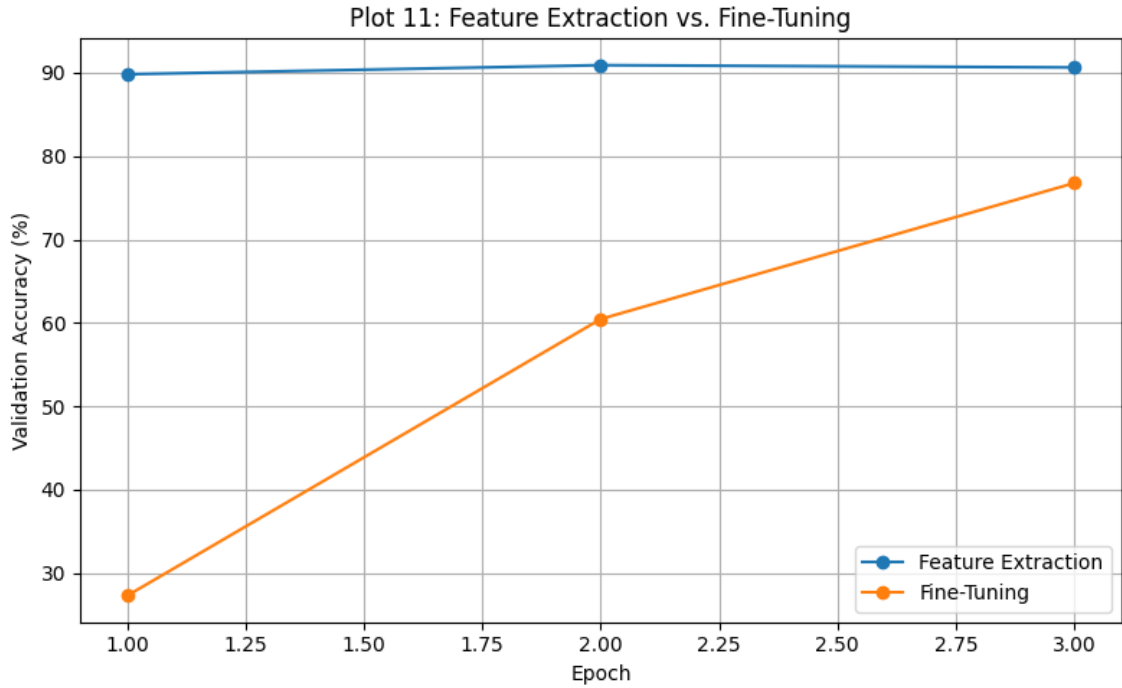


Figure 11: Feature Extraction vs. Fine-Tuning

Inference: The plot compares the validation accuracy obtained using feature extraction and fine-tuning. Feature extraction maintains a high and stable accuracy of around 90%, while fine-tuning improves substantially from approximately 27% to 77% over the three epochs. The lower initial accuracy of fine-tuning occurs because the pretrained layers are being adapted to the new classification task.

Plot 12: Training and Validation Loss

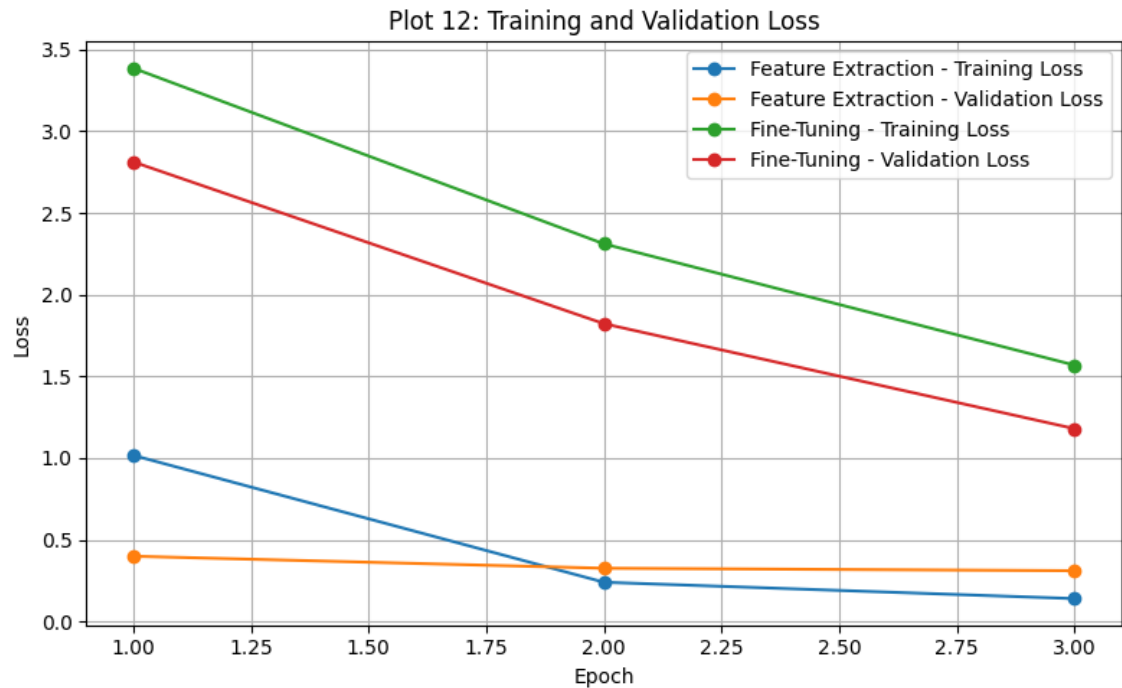


Figure 12: Training and Validation Loss Before and After Fine-Tuning

Inference: The plot shows the training and validation loss for feature extraction and fine-tuning. Both training losses decrease steadily across the epochs, while the validation loss remains relatively stable for feature extraction and decreases for fine-tuning. This indicates that the models are learning effectively, with fine-tuning gradually reducing the loss as the pretrained features are adapted to the dataset.

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

5-Fold Cross-Validation Setup

The dataset was divided into five folds for cross-validation. The total number of samples was 2944. Each fold was used once for validation while the remaining folds were used for training.

- Fold 1: Training = 2355, Validation = 589
- Fold 2: Training = 2355, Validation = 589
- Fold 3: Training = 2355, Validation = 589
- Fold 4: Training = 2355, Validation = 589
- Fold 5: Training = 2356, Validation = 588

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	90.15	88.62	91.51	87.78	88.27	89.27 ± 1.38
C2	89.98	89.30	90.15	87.78	90.65	89.57 ± 1.00
C3	88.79	90.15	91.51	87.61	88.61	89.33 ± 1.36
C4	90.15	88.79	91.85	87.44	88.95	89.44 ± 1.48

Plot 13: 5-Fold Cross-Validation Accuracy

- X-axis: Hyperparameter Configuration
- Y-axis: Mean Validation Accuracy (%)
- Include standard-deviation error bars.

Students should discuss both mean performance and variability.

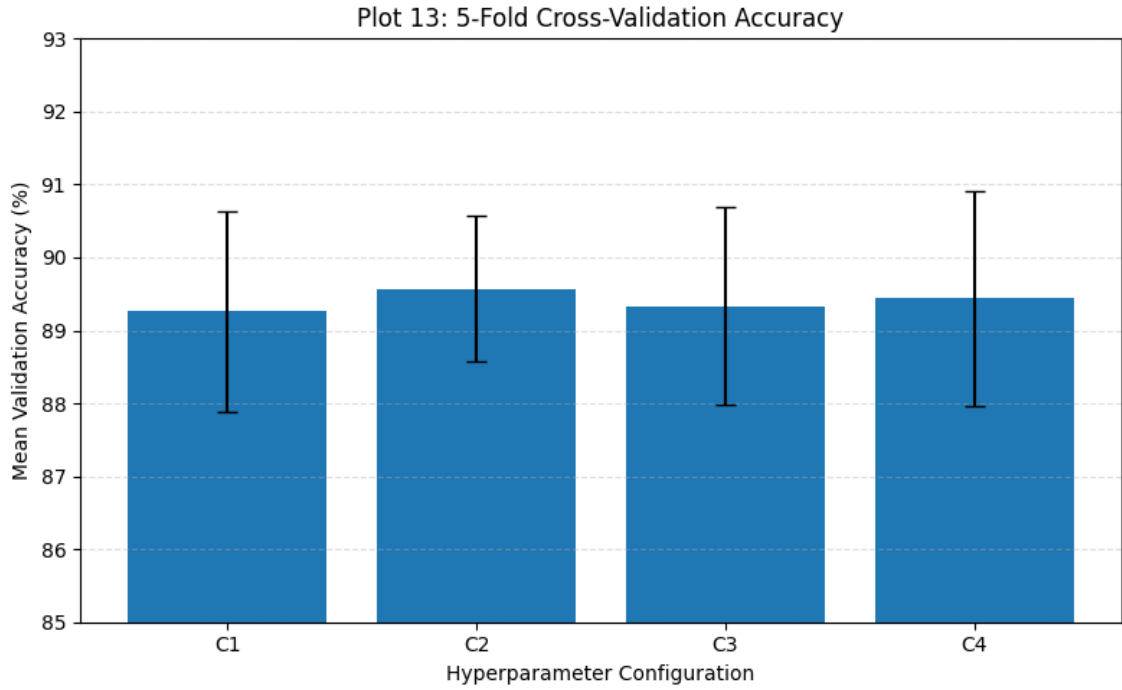


Figure 13: 5-Fold Cross-Validation Accuracy

Inference: The plot compares the mean validation accuracy and standard deviation of the four hyperparameter configurations across five folds. C2 achieves the highest mean accuracy (89.57%) with the lowest standard deviation (1.00%), indicating better and more consistent performance across the folds.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	89.57%
CV Standard Deviation	1.00%
Test Accuracy	87.44%
Precision	88.82%
Recall	87.44%
F1-score	87.19%
Training Time	5.46 seconds
Number of Parameters	168,741

Plot 14: Confusion Matrix

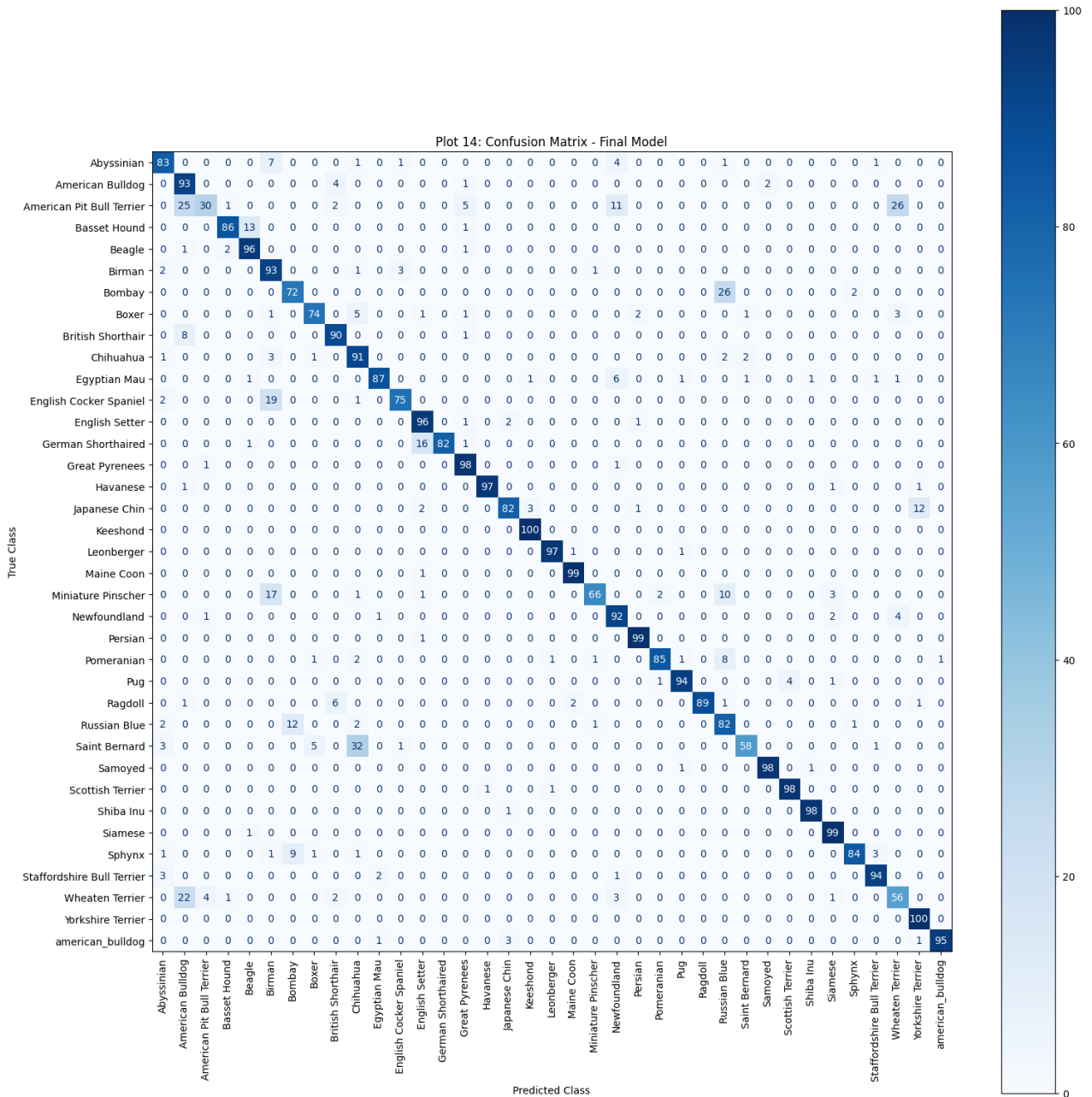


Figure 14: Confusion Matrix of the Final Model

Inference: The confusion matrix shows the classification performance of the final model across all 37 pet breeds. Classes such as Japanese Chin and Wheaten Terrier are classified very well, with predictions concentrated on the diagonal. The most frequent confusions occur between visually similar breeds, such as American Pit Bull Terrier and Staffordshire Bull Terrier, and between Birman and Ragdoll. These errors may occur because related breeds have similar body shapes, fur patterns, colours, and facial features.

Plot 15: Misclassified Images

Plot 15: Representative Misclassified Images

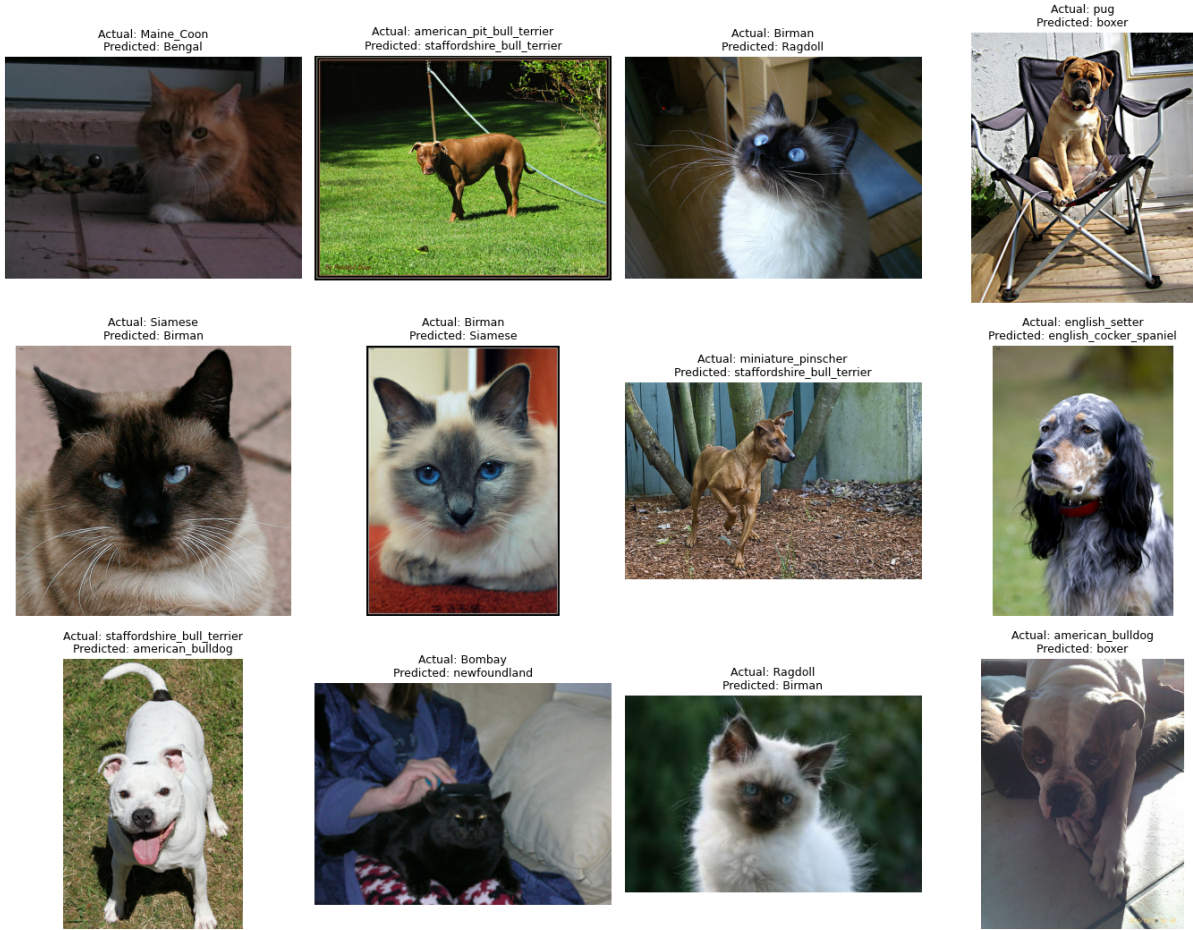


Figure 15: Representative Misclassified Images

Inference: The misclassified examples mainly arise between visually similar breeds. Similar facial structure, coat colour, fur texture, and body shape can make it difficult for the model to distinguish between closely related breeds.

Final Model Discussion: The C2 configuration was selected because it achieved the highest mean cross-validation accuracy of 89.57% with the lowest standard deviation of 1.00%. When retrained using the complete training data, the model achieved a test accuracy of 87.44%, with a precision of 88.82% and an F1-score of 87.19%. The training required only 5.46 seconds and the model contained 168,741 parameters, providing a good balance between classification performance, consistency, and computational cost.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline				
Best Initialization				
Best Regularization				
Best Optimizer				
Best Hyperparameters				
Fine-Tuned Model				

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	N/A	N/A	N/A	N/A
Best Initialization (He)	N/A	N/A	N/A	N/A
Best Regularization	N/A	N/A	N/A	N/A
Best Optimizer (Adam)	N/A	N/A	N/A	90.55 s
Best Hyperparameters (C2)	89.57%	1.00%	87.44%	5.46 s
Fine-Tuned Model	N/A	N/A	N/A	N/A

Configuration	Best Validation Accuracy (%)	Training Time	Remarks
Best Initialization (He)	91.98	—	Highest validation accuracy
Best Regularization	91.17	—	Best validation performance
Best Optimizer (Adam)	91.71	90.55 s	Highest optimizer accuracy
Best Hyperparameters (C2)	90.63	—	Selected for CV
Feature Extraction	90.90	—	Better than fine-tuning
Fine-Tuned Model	76.77	—	Lower validation accuracy

Configuration	Validation Accuracy (%)	Training Time	Final/CV Result
Best Initialization (He)	91.98	—	—
Best Regularization	91.17	—	—
Best Optimizer (Adam)	91.71	90.55 s	—
Best Hyperparameters (C2)	90.63	5.46 s	89.57 $\pm$ 1.00% CV
Feature Extraction	90.90	—	—
Fine-Tuned Model	76.77	—	—

Best Initialization: He initialization achieved the highest validation accuracy of 91.98% among the tested initialization methods.

Best Regularization: No Regularization, Dropout, and Batch Normalization achieved the highest best validation accuracy of 91.17%. Dropout produced the smallest generalization gap of 4.52%.

Best Optimizer: Adam achieved the highest validation accuracy of 91.71%, followed by RMSProp at 90.35%. Adam required 90.55 seconds of training time.

Best Hyperparameters: Configuration C2 achieved the highest mean 5-fold cross-validation accuracy of 89.57% with the lowest standard deviation of 1.00%. The final C2 model achieved 87.44% test accuracy in 5.46 seconds with 168,741 parameters.

Fine-Tuning: Feature extraction achieved a best validation accuracy of 90.90%, whereas the fine-tuned model achieved 76.77% in the recorded experiment.

Overall Justification: Configuration C2 was selected as the final model because it provided the highest mean cross-validation accuracy of 89.57% with the lowest variability of 1.00%. It achieved 87.44% accuracy on the independent test set while requiring only 5.46 seconds of training. Thus, C2 provides a good balance between predictive performance, cross-validation stability, computational cost, and generalization to unseen data.

Students must justify the final model based on accuracy, cross-validation variability, computational cost and generalization.

14. Required Inference for Plots

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?

Each inference should be approximately 2–3 lines.

15. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters are values learned automatically by the model during training, such as weights and biases. Hyperparameters are values selected before training, such as learning rate, batch size, number of epochs, and dropout rate.

2. Why is weight initialization important?

Weight initialization determines the starting values of the trainable weights. A suitable initialization helps the network converge faster and avoids problems such as vanishing or exploding activations.

3. Why can zero initialization be problematic for neural networks?

If all weights are initialized to zero, neurons in the same layer receive the same updates and learn identical features. This breaks symmetry and prevents different neurons from learning useful independent representations.

4. Compare Xavier and He initialization.

Xavier initialization is generally suitable for activation functions such as tanh and sigmoid because it maintains the variance of activations. He initialization is designed mainly for ReLU-type activations and generally gives

better initialization for deep networks using ReLU.

5. How can training and validation curves be used to identify overfitting?

Overfitting can be identified when training performance continues to improve while validation performance stops improving or becomes worse. A growing gap between training and validation accuracy is also an indication of overfitting.

6. How does Dropout reduce overfitting?

Dropout randomly disables a fraction of neurons during training. This prevents the network from depending too strongly on particular neurons and encourages it to learn more robust and distributed features.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes intermediate activations during training. It helps stabilize the learning process, allows faster convergence, and can also provide some regularization.

8. Explain the numerical Batch Normalization example.

In the numerical example, the batch mean and variance are first calculated from the input values. The values are then normalized and scaled and shifted using the learnable parameters γ and β .

9. What are the roles of γ and β ?

The parameter γ controls the scale of the normalized activation, while β controls its shift. Both are learnable parameters that allow the network to choose the appropriate distribution after normalization.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD updates weights using the current gradient, while Momentum also considers previous updates to accelerate learning. RMSProp adapts the learning rate using recent squared gradients, whereas Adam combines adaptive learning rates with momentum and generally provides fast and stable convergence.

11. What happens when the learning rate is too large?

A very large learning rate can cause the optimizer to overshoot the minimum. The loss may fluctuate or diverge instead of decreasing smoothly, resulting in unstable training.

12. What happens when the learning rate is too small?

A very small learning rate causes very small parameter updates. Training then becomes slow and the model may require many epochs to reach a good solution.

13. What is the effect of increasing batch size?

Increasing batch size provides more samples for each parameter update and usually gives more stable gradient estimates. However, very large batches can require more memory and may sometimes reduce the ability of the model to generalize.

14. Explain stride and padding.

Stride specifies the number of pixels by which a convolution filter moves across the input. Padding adds extra pixels around the input, allowing better control of the output size and preserving spatial information near the boundaries.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 is designed as a lightweight CNN using depthwise separable convolutions and inverted residual blocks. These components significantly reduce the number of computations and parameters compared with conventional convolutional networks.

16. What is depthwise separable convolution?

Depthwise separable convolution separates a standard convolution into a depthwise convolution followed by a pointwise 1×1 convolution. This reduces computational cost while still extracting useful spatial and channel features.

17. What is transfer learning?

Transfer learning uses knowledge learned by a model on a large dataset for a new related task. In this experiment, ImageNet-pretrained MobileNetV2 is used to obtain useful image features for classifying pet breeds.

18. Differentiate feature extraction and fine-tuning.

In feature extraction, the pretrained CNN layers are kept frozen and only the new classifier is trained. In fine-tuning, some pretrained layers are unfrozen and updated so that the learned features can adapt to the new dataset.

19. Why is a smaller learning rate generally used during fine-tuning?

Pretrained layers already contain useful features learned from the original dataset. A smaller learning rate allows these weights to be adjusted gradually without destroying the useful representations already learned.

20. Why is K-Fold Cross-Validation useful for hyperparameter selection?

K-Fold Cross-Validation evaluates a configuration using multiple training and validation splits. It provides a more reliable estimate of performance and helps determine whether a configuration performs consistently across different subsets of the training data.

21. Why must the test set remain untouched during tuning?

The test set must remain unseen while selecting models and hyperparameters. Using it during tuning can cause information leakage and produce an overly optimistic estimate of the model's generalization performance.

22. Why should mean and standard deviation both be reported?

The mean shows the average performance across the folds, while the standard deviation indicates how much

the performance varies between folds. A lower standard deviation generally indicates more consistent model performance.

23. Is the highest validation accuracy always sufficient to select a model?

No. Model selection should also consider variability, generalization, computational cost, and other evaluation metrics. A slightly lower accuracy with better stability and generalization may be preferable to a model with the highest validation accuracy alone.

16. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

Source Code and Repository Links

The complete source code, experimental notebooks, plots, and implementation files used in this experiment are provided through the following links.

- **Notebook 1 – Main CNN / MobileNetV2 Notebook:** Open Notebook
- **Notebook 2 – Cross-Validation Notebook:** Open Notebook
- **GitHub Repository:** Open GitHub Repository

17. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

18. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks,” CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, “Cats and Dogs,” IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>